

Practical-7

Aim: Implementation of making a change problem using dynamic programming.

Making a change problem

Algorithm:

Algorithm:-

Input : No of coins n

coin denominations $\text{coins}[0..n-1]$

Amount to be changed amount.

Output : minimum no. of coins required to make the given amount.

Step 1: Start

Step 2: Read the no. of coins n

Step 3: Read the coin denominations

Step 4: Read the required amount

Step 5: Create a DP array $\text{dp}[\text{amount}]$

Step 6: Set $\text{dp}[0] = 0$

Step 7: for each amount i from 1 to amount

- set $\text{dp}[i] = 32767$

- check every coin denomination

- if $\text{coin}[i] \leq i$, calculate

$\text{dp}[i] = \min[\text{dp}[i], \text{dp}[i - \text{coin}[i]] + 1]$

After 8: After filling the DP table $\text{dp}[\text{amount}]$ contains the minimum no. of coins

Step 9: Display $\text{dp}[\text{amount}]$

Step 10: Stop.



Program:

```
#include <iostream>
using namespace std;

int main()
{
    int n, amount;

    cout << "Enter number of coins: ";
    cin >> n;

    int coin[n];

    cout << "Enter coin denominations: ";
    for (int i = 0; i < n; i++)
    {
        cin >> coin[i];
    }

    cout << "Enter amount: ";
    cin >> amount;

    int dp[amount + 1];

    dp[0] = 0;

    for (int i = 1; i <= amount; i++)
    {
        dp[i] = 32767;

        for (int j = 0; j < n; j++)
        {
            if (coin[j] <= i)
            {
                dp[i] = min(dp[i], dp[i - coin[j]] + 1);
            }
        }
    }

    cout << "Minimum number of coins required = "
```

```
<< dp[amount] << endl;  
  
return 0;  
}
```

Output:

Output

```
Enter number of coins: 6  
Enter coin denominations: 1 3 5 7 9 10  
Enter amount: 17  
Minimum number of coins required = 2  
  
=== Code Execution Successful ===
```

Time Complexity:

Best Case:

Time complexity:
Best case:
for($i=2; i \leq \text{amount}; i++$) $\rightarrow$ amount time
for($j=0; j < n; j++$) $\rightarrow n$
if($\text{coin}[j] \leq i$) $\rightarrow 2$
 $\text{dp}[i] = \min(-)$ $\rightarrow 2$
 $T(n) = \text{amount} \times n$
 $= O(\text{amount} \times n)$.

Average Case:

Average case:
for ($i=1$; $i \leq \text{amount}$; $i++$) $\rightarrow \text{amount}$
for ($j=0$; $j < n$; $j++$) $\rightarrow n$
if ($\text{coin}[j] \leq i$) $\rightarrow 1$
 $\text{dp}[i] = \min()$ $\rightarrow 1$
 $T(n) = \text{amount} \times n$
 $T(n) = O(\text{amount} \times n)$

Worst Case:

Worst case:
for ($i=1$; $i \leq \text{amount}$; $i++$) $\rightarrow \text{amount}$
for ($j=0$; $j < n$; $j++$) $\rightarrow n$
if ($\text{coin}[j] \leq i$) $\rightarrow 1$
 $\text{dp}[i] = \min()$ $\rightarrow 1$
 $T(n) = \text{amount} \times n$
 $T(n) = O(\text{amount} \times n)$



Conclusion:

The implementation and time analysis of the Making Change Problem using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers different coin denominations to find the minimum number of coins required to make the given amount. It has a time complexity of $O(n \times \text{amount})$ in the best, average, and worst cases.

Therefore, Dynamic Programming provides an efficient and systematic solution for solving the Making Change Problem.